Built-in models (`m.models`)

High-level interface for turnkey Bayesian models in BayesForge: DPMM, GMM, PCA, Survival analysis, BNNC, and NBDA.

DPMM (`m.models.dpmm`)

Dirichlet Process Mixture Models with truncated stick-breaking representation and consensus clustering.

`m.models.dpmm.dpmm_latent`

```
m.models.dpmm.dpmm_latent(data, T=10, alpha=None)
```

Latent Variable formulation: Explicitly samples 'z'. Requires a sampler that supports discrete variables (e.g., MixedHMC or DiscreteHMCGibbs).

`m.models.dpmm.dpmm_marginal`

```
m.models.dpmm.dpmm_marginal(data, T=10, alpha=None)
```

Marginalized formulation: Integrates out 'z'. Standard formulation for NUTS/HMC samplers.

`m.models.dpmm.get_cluster_probs`

```
m.models.dpmm.get_cluster_probs(data, w, mu, sigma, Lcorr)
```

Get the cluster probabilities for each data point and each sample.

Argument	Type	Description
`data`		Data array (N, D).
`w`		Mixture weights (N_samples, N_components).
`mu`		Cluster means (N_samples, N_components, D).
`sigma`		Scale (variance) parameters (N_samples, N_components, D).
`Lcorr`		Cholesky correlation matrices (N_samples, N_components, D, D).

Returns

- Cluster probabilities (N_samples, N_data, N_components).

`m.models.dpmm.mix_weights`

```
m.models.dpmm.mix_weights(beta)
```

Mixture weights (stick-breaking) for DPMM. The stick-breaking weights are used to sample the mixture component assignments. Args: beta: Stick-breaking weights (T-1 components) Returns: w: Mixture weights (T components) Reference: https://pyro.ai/examples/dirichlet_process_mixture.htm

`m.models.dpmm.model`

```
m.models.dpmm.model(data, T=10, method='marginal', alpha=None)
```

Wrapper function for DPMM.

Argument	Type	Description
`data`		Input data array (N, D)
`T`		Truncation level (max number of clusters)
`method`		'marginal' (default because it is faster, for NUTS) or 'latent' (explicit z, requires MixedHMC/Gibbs)
`alpha`		Stick-breaking parameter (default: None)

`m.models.dpmm.plot`

```
m.models.dpmm.plot(
    data,
    sampler,
    figsize=(10, 8),
    point_size=10,
    credible=(0.25, 0.5, 0.75, 0.95),
    show_density=False,
    density_levels=7,
    spread_cap=2.0,
    title=None,
    threshold=0.5,
    linkage_method='average',
)
```

Posterior view of the DPMM clusters.

Instead of a single posterior-predictive contour (whose lowest auto-selected level is a tiny positive number that Matplotlib prints as `0.00` and which can bridge unrelated clusters through the wide, near-empty components of the truncation), this draws **one credible region per consensus cluster**:

1. `predict` performs the consensus clustering on the soft assignments, so the cluster labels are label-switching safe.
2. For each cluster, every `(draw, component)` pair is weighted by the soft mass that component assigns to that cluster, and the spread of the resulting mixture is moment-matched to a single Gaussian **centered on the cluster's centroid** (anchoring on the mixture mean instead lets overlapping components that swap identities across draws stretch the ellipse across the plot).

3. The Gaussian is drawn as an ellipse at each probability in `credible` (2D Gaussian quantiles `-2 ln(1 - p)`).

Argument	Type	Description
`data`		Data array (N, D).
`sampler`		Fitted BF sampler.
`figsize`		Figure size.
`point_size`		Scatter point size.
`credible`		Iterable of credible-mass levels for the ellipses, e.g. `[0.5, 0.9]` or `[0.25, 0.5, 0.75, 0.95]`. Values are probabilities in `(0, 1)`; one nested ellipse is drawn per level (largest level solid, others dashed). Defaults to :data:`DEFAULT_CREDIBLE` (ten nested levels, 5%-95%).
`show_density`		If True, overlay the posterior-predictive density with explicit *relative* levels (no misleading `0.00`).
`density_levels`		Number of relative density levels when `show_density` is True.
`spread_cap`		Each ellipse axis is capped at this multiple of the cluster's own point-cloud std (see `_cap_covariance`). Set to `None` for the uncapped moment-matched ellipse.
`title`		Custom title for the plot. If None, uses default summary title.
`threshold`		Distance threshold for consensus clustering cut.
`linkage_method`		Linkage criterion ('average', 'complete', etc.).

m.models.dpmm.predict

```
m.models.dpmm.predict(
    data,
    sampler,
    return_probs=False,
    threshold=0.5,
    linkage_method='average',
)
```

Performs Consensus Clustering from the posterior samples of the DPMM.

This function extracts the posterior samples from a Dirichlet Process Mixture Model (DPMM) and derives a single, consensus clustering partition of the data points. The consensus procedure operates on the Posterior Similarity Matrix (PSM) and is strictly invariant to label switching across MCMC chains.

Methodology: 1. **Soft Assignment Probabilities**: For each posterior draw (subsampled to an upper limit for memory efficiency), the soft responsibility vector of each data point across all mixture components is computed: `p(z_i = k | x_i, theta) proportional to w_k * N(x_i | mu_k, Sigma_k)`.

2. **Posterior Similarity Matrix (PSM)**: The pairwise co-clustering probability matrix is computed by averaging the inner product of soft assignments over all draws: `S_ij = E_{posterior}[sum_k p(z_i = k) * p(z_j = k)]`. Entry `(i, j)` represents the posterior probability that point `i` and point `j` belong to the same cluster.
3. **Distance Conversion**: The similarity matrix is converted into a metric distance matrix `D_ij = 1 - S_ij`, bounded in `[0, 1]` with zero diagonal.
4. **Agglomerative Hierarchical Clustering**: Agglomerative clustering is applied to the condensed distance matrix using the specified linkage criterion (e.g. `'average'`, `'complete'`). The resulting dendrogram is cut at cophenetic distance `threshold` via `scipy.cluster.hierarchy.fcluster` to produce the final cluster labels.

Argument	Type	Description
`data`		Input data array of shape `(N, D)`.
`sampler`		Fitted BayesForge sampler instance holding DPMM posterior samples.

Argument	Type	Description
`return_probs`	bool	If True, returns additional raw posterior draws and the per-draw soft assignment tensor `cluster_probs` of shape `(N_draws, N_data, T)`, needed by `plot` to compute credible ellipses per consensus cluster. Defaults to False.
`threshold`	float	Cophenetic distance threshold in `[0, 1]` used to cut the hierarchical tree (criterion `'distance'`). A smaller threshold yields more fine-grained clusters, while a larger threshold merges nearby clusters. Defaults to 0.5.

Argument	Type	Description
`linkage_method`	str	Linkage criterion used for agglomeration. Supported methods include: - `'average'`: UPGMA. Merges clusters based on the average pairwise distance. Good general default, but can exhibit chaining in elongated clouds. - `'complete'`: Maximum/farthest-neighbor linkage. Avoids chaining and produces compact, well-separated clusters; recommended when clusters are close together. - `'single'`: Minimum/nearest-neighbor linkage. Highly prone to chaining; useful only for non-spherical nested manifolds. - `'weighted'`: WPGMA. Gives equal weight to merging clusters regardless of their size. - `'centroid'`: UPGMC. Distance between geometric centroids. - `'median'`: WPGMC. Distance between weighted centroids. - `'ward'`: Minimizes the total within-cluster variance. Requires Euclidean distance geometry. Defaults to `'average'`.

Returns

- If `return_probs` is False: Tuple `(w, mu, sigma, Lcorr, final_labels)`: - `w`: Posterior mixture weights draws (subsampled), shape `(S, T)`. - `mu`: Posterior component means draws, shape `(S, T, D)`. - `sigma`: Posterior scale vectors draws, shape `(S, T, D)`.

- `Lcorr`: Posterior correlation Cholesky factors, shape `(S, T, D, D)`. - `final_labels`: 1D array of length `N` with integer cluster labels.
- If `return_probs` is True: Tuple `(w, mu, sigma, Lcorr, final_labels, cluster_probs)`, where `cluster_probs` has shape `(S, N, T)`.

References

- Fritsch, A. & Ickstadt, K. (2009). "Improved criteria for clustering based on the posterior similarity matrix". Bayesian Analysis, 4(2), 367-391.
- Dahl, D. B. (2006). "Model-Based Clustering for Expression Data via a Dirichlet Process Mixture Model". In Bayesian Inference and Methods in Investigative Genomics.
- Wade, S. & Ghahramani, Z. (2018). "Bayesian cluster analysis: Point estimation and credible balls". Bayesian Analysis, 13(2), 559-626.

`m.models.dpmm.proportion_of_data_assigned_to_cluster`

```
m.models.dpmm.proportion_of_data_assigned_to_cluster()
```

Plots the proportion of data assigned to clusters, sorted by size.

Fixes the Label Switching issue in boxplots by visualizing "1st Largest Cluster", "2nd Largest", etc., instead of "Index 0", "Index 1".

GMM (`m.models.gmm`)

Gaussian Mixture Models with fixed components and posterior co-clustering / credible ellipses.

`m.models.gmm.plot`

```
m.models.gmm.plot(
    data,
    sampler,
    figsize=(10, 8),
    point_size=15,
    credible=(0.25, 0.5, 0.75, 0.95),
    show_density=False,
    density_levels=7,
```

```
    spread_cap=2.0,
)
```

Posterior view of the GMM clusters.

Same visualization as `dpmm.plot`: one credible ellipse per consensus cluster (labels made switching-safe by the posterior similarity matrix), each axis capped at `spread_cap` times the cluster's own point spread. Replaces the old density contour whose lowest auto-selected level printed as a misleading `0.00`.

Argument	Type	Description
`data`		Data array (N, D).
`sampler`		Fitted BF sampler.
`figsize`		Figure size.
`point_size`		Scatter point size.
`credible`		Iterable of credible-mass levels for the ellipses, e.g. `[0.5, 0.9]` or `[0.25, 0.5, 0.75, 0.95]`. One nested ellipse per level (largest solid, others dashed). Defaults to :data:`DEFAULT_CREDIBLE` (ten nested levels, 5%-95%).
`show_density`		If True, overlay the posterior-predictive density with explicit *relative* levels (no misleading `0.00`).
`density_levels`		Number of relative density levels when `show_density` is True.
`spread_cap`		Ellipse axis cap as a multiple of the cluster's point std (see `dpmm._cap_covariance`). None to disable.

`m.models.gmm.predict`

```
m.models.gmm.predict(data, sampler, return_probs=False)
```

Predicts the GMM density contours based on posterior samples and final labels.

This function processes the posterior samples from a Bayesian Gaussian Mixture Model (GMM) to derive a single, representative clustering and the corresponding GMM parameters. The approach is based on two main steps: calculating posterior point estimates and performing co-clustering analysis via a Posterior Similarity Matrix (PSM).

Methodology: 1. **Posterior Point Estimates**: The function first computes the posterior mean of the GMM parameters (weights, means, and covariance components) by averaging over all samples obtained from the MCMC sampler. This provides a single set of parameters that represents the central tendency of the posterior distribution.

2. **Co-clustering via Posterior Similarity Matrix (PSM)**: To obtain a single, robust clustering, this function uses a co-clustering approach that leverages the entire posterior distribution.

    a. A similarity matrix is constructed where the entry (i, j) represents the posterior probability that data point `i` and data point `j` belong to the same cluster. This is calculated by averaging their co-assignment probabilities across all posterior samples.
    b. This similarity matrix is converted into a distance matrix (1 - similarity).
    c. Agglomerative hierarchical clustering ('average' linkage) is then applied to this distance matrix.
    d. The resulting hierarchy is cut at a specified distance threshold to produce a final, single set of cluster labels for the data.

This method avoids issues like label switching inherent in MCMC and produces a stable clustering that summarizes the relationships discovered in the full Bayesian posterior.

- data (jnp.ndarray): The input data points used for the model, with shape [N, D] where N is the number of samples and D is the number of features.
- sampler (BF.sampler): The fitted MCMC sampler object from which posterior samples can be extracted.
- tuple: A tuple containing four elements:
    - post_mean_w (jnp.ndarray): The posterior mean of the mixture weights.
    - post_mean_mu (jnp.ndarray): The posterior mean of the cluster means.
    - post_mean_cov (jnp.ndarray): The reconstructed posterior mean of the cluster covariance matrices.
    - final_labels (jnp.ndarray): The final, single-partition cluster labels for each data point. Plus `(w_samps, mu_samps, sigma_samps, Lcorr_samps, cluster_probs)` when `return_probs` is True (raw draws + per-draw soft assignments, used by `plot_gmm` for the credible ellipses).

Reference: * https://www.pure.ed.ac.uk/ws/portalfiles/portal/80251963/1508378464.pdf * https://arxiv.org/abs/2108.11753 * https://en.wikipedia.org/wiki/Mixture_model * https://projecteuclid.org/journals/bayesian-analysis/volume-4/issue-2/Improved-criteria-for-clustering-based-on-the-posterior-similarity-matrix/10.1214/09-BA414.pdf * https://www.pure.ed.ac.uk/ws/por * https://pmc.ncbi.nlm.nih.gov/articles/PMC10441802/

PCA (`m.models.pca`)

Probabilistic Principal Component Analysis (classic, ARD, robust Student-t, sparse Laplace/horseshoe).

`m.models.pca.align_posterior_samples`

```
m.models.pca.align_posterior_samples(posterior_w, reference_components)
```

Aligns each posterior sample to the reference component matrix using Procrustes analysis. Args: posterior_w: Array of shape (num_samples, data_dim, latent_dim) reference_components: Array of shape (data_dim, latent_dim) Returns: aligned_w: Array of shape (num_samples, data_dim, latent_dim)

`m.models.pca.analyze_aligned_posteriors`

```
m.models.pca.analyze_aligned_posteriors(X_scaled, aligned_w_samples)
```

Calculates final PCA attributes using a self-contained, deterministic sign convention for the components. Args: X_scaled: Scaled data matrix of shape (num_datapoints, data_dim) aligned_w_samples: Aligned posterior samples of shape (num_samples, data_dim, latent_dim) Returns: Dictionary with keys for components, variance, etc.

`m.models.pca.components`

```
m.models.pca.components(feature_names, bayesian_results=None, ax=None)
```

Plots the final Bayesian component matrix as a heatmap.

m.models.pca.confidence_ellipse

```
m.models.pca.confidence_ellipse(x, y, ax, n_std=2.0, facecolor='none', **kwargs)
```

Create a confidence ellipse for a set of x, y points.

m.models.pca.create_reference_from_posterior

```
m.models.pca.create_reference_from_posterior(posterior_w)
```

Creates a reference component matrix from the posterior samples using SVD. This reference will be used to align all posterior samples. Args: posterior_w: Array of shape (num_samples, data_dim, latent_dim) Returns: reference_components: Array of shape (data_dim, latent_dim)

m.models.pca.cumulative_variance

```
m.models.pca.cumulative_variance(
    X=None,
    bayesian_results=None,
    figsize=(16, 8),
    alpha=0.7,
    fontsize=10,
    color_bar='blue',
    color_cumulative_line='red',
    label_size=16,
    ax=None,
)
```

Plots the cumulative explained variance for Bayesian PCA.

m.models.pca.get_model

```
m.models.pca.get_model(type='ARD')
```

Principal Component Analysis (PCA) model using JAX and BF. Args: type (str): Type of PCA model to use. Options are 'ARD', 'robust', 'sparse', 'sparse_robust_ard', 'classic'. Returns: PCA model class using JAX and BF.

m.models.pca.individual_correlation

```
m.models.pca.individual_correlation(
    X=None,
    bayesian_results=None,
    y=None,
    feature_names=None,
    target_names=None,
    color_var=None,
    size_var=None,
    shape_var=None,
    ax=None,
    figsize=(10, 10),
)
```

Generates a variable correlation plot with cos2 values for Bayesian PCA results.

m.models.pca.model_sparse_robust_ard

```
m.models.pca.model_sparse_robust_ard(X, data_dim, latent_dim, num_data_points)
```

A combined Sparse, Robust Bayesian PCA model with Automatic Relevance Determination (ARD).

- Sparsity: Achieved with a Laplace prior on the weights 'w'.
- Robustness: Achieved with a Student's t-distribution for the likelihood.
- ARD: Achieved by placing a hierarchical prior on the scale of the Laplace distribution, allowing entire latent dimensions to be pruned.

m.models.pca.pca_ARD

```
m.models.pca.pca_ARD(X, data_dim, latent_dim, num_data_points)
```

Probabilistic PCA model with ARD (Automatic Relevance Determination) prior on weights and a correctly defined noise model. ARD helps in automatic relevance determination of latent dimensions, by allowing the model to learn which dimensions are important. Args: x_train: Observed data matrix of shape (data_dim, num_datapoints). data_dim: Dimensionality of the observed data. latent_dim: Dimensionality of the latent space. num_data_points: Number of data points.

m.models.pca.pca_classic

```
m.models.pca.pca_classic(X, data_dim, latent_dim, num_data_points)
```

Classic Probabilistic PCA model with standard isotropic Gaussian priors.

Parameters: - X: Observed data matrix (shape: [num_data_points, data_dim]). - data_dim: Number of features / observed dimensions. - latent_dim: Number of latent principal components. - num_data_points: Number of observations.

m.models.pca.pca_robust

```
m.models.pca.pca_robust(X, data_dim, latent_dim, num_data_points)
```

Robust Bayesian PCA model using a Student's t-distribution for the likelihood.

m.models.pca.pca_sparse

```
m.models.pca.pca_sparse(X, data_dim, latent_dim, num_data_points)
```

Sparse Bayesian PCA model using a Laplace prior on the weights (w).

m.models.pca.plot

```
m.models.pca.plot(
    X=None,
    bayesian_results=None,
    y=None,
    feature_names=None,
    target_names=None,
    color_var=None,
    size_var=None,
    shape_var=None,
    figsize=(22, 18),
)
```

Generates a 2x2 dashboard of the four main PCA plots. Args: X (jnp.ndarray, optional): Scaled data matrix. Defaults to self.X. feature_names (list, optional): List of names for the original features. Required for some plots. y (jnp.ndarray, optional): Grouping variable for the individuals plot. target_names (list, optional): Names for the groups in y. figsize (tuple, optional): Overall size of the 2x2 plot.

m.models.pca.process_var

```
m.models.pca.process_var(var)
```

Convert categorical string array to numeric codes; leave numeric as is.

m.models.pca.set_deterministic_sign

```
m.models.pca.set_deterministic_sign(components)
```

Enforces a deterministic sign convention on the component matrix. For each component (column), it finds the element with the largest absolute value and flips the sign of the entire component if that element is negative. Args: components: The component matrix of shape (data_dim, latent_dim) Returns: The component matrix with a consistent sign convention.

m.models.pca.variable_correlation

```
m.models.pca.variable_correlation(feature_names, X=None, bayesian_results=None, ax=None)
```

Generates a variable correlation plot with cos2 values.

Survival (`m.models.survival`)

Discrete-time Bayesian survival and hazard rate models with univariate, multivariate, and time-varying covariates.

`m.models.survival.build_surv_covariates`

```
m.models.survival.build_surv_covariates()
```

Build a full covariate array for survival modeling.

- Fixed covariates (N,) are expanded to (N,T)
- Stacked with existing time-varying covariates (N, T, V)
- Updates covariate names with fixed covariates first

`m.models.survival.cum_hazard`

```
m.models.survival.cum_hazard(hazard)
```

Calculates the cumulative hazard from a given hazard rate.

Parameters hazard : numpy.ndarray or jax.numpy.ndarray A 1D or 2D array representing the hazard rate at each time step.

Returns numpy.ndarray or jax.numpy.ndarray The cumulative hazard calculated by summing the hazard over time steps.

Notes - The cumulative hazard is computed as the cumulative sum of the hazard values, scaled by the `interval_length` factor.

`m.models.survival.import_covF`

```
m.models.survival.import_covF(cov, names)
```

Import fixed covariates (patient-level).

m.models.survival.import_covV

```
m.models.survival.import_covV(cov, names)
```

Import time-varying covariates.

Parameters cov : np.ndarray Array of shape (N, T) for one covariate, or (N, T, V) for multiple covariates. names : list of str Names of the covariates.

m.models.survival.import_time_even

```
m.models.survival.import_time_even(time, event, interval_length=1)
```

Convert data into of survival information into a surv object. Converts continuous time and event data into discrete time intervals for survival analysis if interval_length > 1.

Parameters time : 1D jnp.ndarray of shape (N,) representing the continuous time variable. event : 1D Binary jnp.ndarray of shape (N,) representing the event indicator. interval_length : int, optional The length of each discrete time interval (default is 3).

Returns - n_patients : int Number of patients in the dataset - patients : np.array Array of patient indices - time : np.array Array of time points - event : np.array Array of event status - interval_bounds : jnp.ndarray Array of boundaries for discrete time intervals. - n_intervals : int The total number of discrete intervals. - intervals : jnp.ndarray Array of interval indices. - death : jnp.ndarray A binary matrix (n_patients x n_intervals) indicating whether each subject experienced the event in each interval (1 if the event occurred, 0 otherwise). - exposure : jnp.ndarray A matrix (n_patients x n_intervals) indicating the time each subject was exposed in each interval.

Notes - The function assumes that subjects who experienced the event did so at the end of the time period. - Exposure is capped by the interval bounds, and the last interval reflects the remaining time to the event or censoring.

m.models.survival.model

```
m.models.survival.model(death, cov, exposure, censoring=None)
```

Bayesian Survival Analysis model dispatching to univariate, multivariate, or time-varying models.

Parameters: - death: Event occurrence indicator array. - cov: Covariates array (fixed or time-varying). - exposure: Exposure times array across intervals. - censoring: Optional censoring indicators.

m.models.survival.model_multivariate

```
m.models.survival.model_multivariate(death, cov, exposure, censoring=None)
```

Multivariate discrete-time survival model with multiple fixed covariates.

Parameters: - death: Event occurrence indicator array. - cov: 2D covariates array (observations, covariates). - exposure: Exposure times array across intervals. - censoring: Optional censoring indicators.

m.models.survival.model_time_varying

```
m.models.survival.model_time_varying(death, cov, exposure, censoring=None)
```

Time-varying discrete-time survival model with longitudinal covariates.

Parameters: - death: Event occurrence indicator array. - cov: 3D time-varying covariates array (observations, intervals, covariates). - exposure: Exposure times array across intervals. - censoring: Optional censoring indicators.

m.models.survival.model_univariate

```
m.models.survival.model_univariate(death, cov, exposure, censoring=None)
```

Univariate discrete-time survival model with a single fixed covariate.

Parameters: - death: Event occurrence indicator array. - cov: Single covariate array. - exposure: Exposure times array across intervals. - censoring: Optional censoring indicators.

m.models.survival.observed

```
m.models.survival.observed(observed)
```

Add observed data to the survival object. Args: observed (np.ndarray): Array of observed data with shape (N, T). This array represent if individuals have been observed at each time point and thus should be 1 if observed and 0 if censored. Raises: ValueError: If observed data shape does not match the survival object.

m.models.survival.plot_censoring

```
m.models.survival.plot_censoring(
    cov=None,
    event='event',
    time='time',
    xlabel='Time',
    ylabel='Subject',
)
```

Plots the censoring status of subjects in a time-to-event dataset.

Uses the arrays populated by `import_time_even` (`self.time`, `self.event`, `self.patients`) and, when `cov` names a fixed covariate imported via `import_covF`, overlays the subjects for which that covariate equals 1.

Parameters cov : str, optional Name of a fixed covariate (as passed to `import_covF`) to overlay as scatter points. If `None` or not found, no overlay is drawn. event, time : str, optional Kept for backwards compatibility; unused. xlabel : str, optional Label for the x-axis. Default is 'Time'. ylabel : str, optional Label for the y-axis. Default is 'Subject'.

Returns (fig, ax) The Matplotlib figure and axis showing censored and uncensored subjects along with the specified covariate overlay.

m.models.survival.plot_surv

```
m.models.survival.plot_surv(
    lambda0='Baseline_rate',
    beta='beta',
    xlab='Time',
```

```
    ylab='Survival',
    covlab='treated',
    title='Bayesian survival model',
    hdi_prob=0.94,
)
```

Plot posterior cumulative hazard Lambda(t) and survival function S(t).

Draws two subplots: the cumulative hazard on the left and the survival curves on the right, with equal-tailed credible bands.

Parameters: - lambda0: Posterior variable name for baseline rate draws. - beta: Posterior variable name for covariate effect draws. - xlab: Label for the x-axis. - ylab: Label for the y-axis. - covlab: Label for covariate groups. - title: Super-title for the figure. - hdi_prob: Credible interval probability mass (defaults to 0.94).

Returns: - Tuple of (fig, (hazard_ax, surv_ax)).

`m.models.survival.ppc_km_by_cov`

```
m.models.survival.ppc_km_by_cov(
    covs=None,
    n_groups=3,
    n_draws=None,
    seed=0,
    n=50,
    extrapolation_factor=1.2,
    risk=True,
    lambda0='Baseline_rate',
    beta='beta',
    facet=False,
)
```

Grouped KM-overlay PPCs of the replicated times, one figure per covariate.

Builds on `sample_times()` and `m.diag.ppc_km_overlay(group=...)`: each covariate groups the patients – at most `n_groups` distinct values are grouped as-is, anything more is binned into `n_groups` quantile bins (Q1..Qn) – so a multivariate model gets one bayesplot-style grouped KM check per covariate instead of a single pooled curve.

By default (`facet=False`), each figure overlays the survival curves of all covariate levels on the same plot with distinct colors per level. Setting `facet=True` separates the levels into subplots (one panel per level).

With `risk=True` the dict also holds a "risk" figure grouping the patients by quantiles of the posterior-mean linear predictor: one panel per quantile of predicted hazard, which checks all covariates at once (Q1 = lowest predicted hazard).

Argument	Type	Description
`covs`		Covariate name, list of names, or None for all of `self.cov_names`.
`n_groups`		Max number of distinct values grouped as-is, and number of quantile bins otherwise.
`n_draws, seed`		Passed to `sample_times`.
`n, extrapolation_factor`		Passed to `ppc_km_overlay`.
`risk`		Also build the linear-predictor risk-group figure.
`lambda0, beta`		Posterior names, as in `sample_times`.
`facet`		When False (the default), overlays groups on a single plot with distinct colors. When True, separates them into subplots (one panel per level).

Returns: dict {label: plotly Figure} keyed by covariate name, plus "risk" when `risk=True`.

`m.models.survival.priors`

```
m.models.survival.priors(sample=False)
```

Define prior distributions for the survival model.

Parameters: - sample: If True, draws direct prior samples rather than defining model nodes.

Returns: - Tuple of (lambda0, beta) priors for baseline hazard and covariate effects.

`m.models.survival.sample_times`

```
m.models.survival.sample_times(
    lambda0='Baseline_rate',
    beta='beta',
    n_draws=None,
    seed=0,
)
```

Posterior predictive event times, one row per posterior draw.

Returns a `(n_draws, n_patients)` numpy array of replicated event times. For each posterior draw the piecewise-exponential rate is `rate[s, i, k] = lambda0[s, k] * exp(cov[i] @ beta[s])` and the event time is the exponential draw inverted through its cumulative hazard.

The interval indicators in `yrep` are not durations, so this is what a bayesplot-style KM overlay needs: `m.diag.ppc_km_overlay(y=s.time, yrep=s.sample_times(), status_y=s.event)`. Like bayesplot's predicted curves these times carry no censoring: a draw beyond the last interval bound is extrapolated with the last interval's rate, and a zero rate gives `np.inf`. For one panel per covariate see `ppc_km_by_cov`.

`m.models.survival.survival`

```
m.models.survival.survival(hazard)
```

Calculates the survival probability from a given hazard rate.

Parameters hazard : numpy.ndarray or jax.numpy.ndarray A 1D or 2D array representing the hazard rate at each time step.

Returns numpy.ndarray or jax.numpy.ndarray The survival probability, computed as the exponential of the negative cumulative hazard.

Notes - Survival is calculated as `exp(-cumulative hazard)` where the cumulative hazard is calculated using the `cum_hazard` function.

BNNC (`m.models.bnnc`)

Bayesian Neural Network Clustering for deep unsupervised representation and clustering.

m.models.bnnc.get_cluster_probs

```
m.models.bnnc.get_cluster_probs(data_pts, theta_prob, mu_p, sigma_p)
```

Compute normalized posterior cluster membership probabilities for data points.

Parameters: - data_pts: Data array (shape: [N, D]). - theta_prob: Cluster mixture weights (shape: [K] or [N_draws, K]). - mu_p: Cluster component means (shape: [K, D]). - sigma_p: Cluster component standard deviations (shape: [K, D]).

Returns: - Array of normalized posterior cluster assignment probabilities (shape: [N, K]).

m.models.bnnc.model

```
m.models.bnnc.model(data, K=11, D_H1=10, empirical_bayes=False)
```

BNNC marginal mixture model definition.

m.models.bnnc.plot

```
m.models.bnnc.plot(data, sampler, figsize=(10, 8), point_size=30)
```

Posterior predictive plot: a filled contour of the expected cluster index over the feature plane, with the data coloured by their argmax cluster.

The predictive at a point x is p(k | x) pi_k * N(x | mu_k, sigma_k), averaged over posterior draws of (global_pi, mu, sigma). The contour shows sum_k k * p(k | x); the colour bar is that expected index.

m.models.bnnc.predict

```
m.models.bnnc.predict(data, sampler)
```

Performs Consensus Clustering.

NBDA (`m.models.nbda`)

Network-Based Diffusion Analysis separating social and asocial transmission processes.

`m.models.nbda`

```
m.models.nbda(CES=True)
```

Network-Based Diffusion Analysis (NBDA) probabilistic model.

- CES: If True, uses the Constant Elasticity of Substitution (CES) parameterization to combine social and asocial transmission rates.

Returns: - Probabilistic model sampling diffusion events conditional on network structure and status.